

Gunter & Scott 1990 — Inventory of Definitions and Theorems

Source: **C. A. Gunter and D. S. Scott**, “**Semantic Domains**,” *Handbook of Theoretical Computer Science* Vol. B, North-Holland, 1990, pp. 633–674 ([../papers/Gunter Scott 1990.pdf](#)).

The work list for the Lean formalization: every definition and every one of the paper’s **30 numbered results** (Theorems / Lemmas / Proposition 1–30), in paper order, matched to its Lean equivalent.

Progress (as of r0032, 2026-0806)

#	Quantity	Done	Remaining	Of
1	Definitions to define	all ≈ 13 — the computable function landed in r0031; see the note below on D^∞	0	≈ 13
2	Numbered results complete	19 (Thm 1, Thm 3, Lem 4, Lem 5, Thm 6, Thm 7, Lem 8, Lem 10, Thm 11, Lem 13, Prop 15, Lem 17, Lem 19, Lem 20, Thm 21, Thm 22, Lem 23, Lem 24 , Thm 25)	9	29
2a	— resolved by refutation	Thm 16 — the algebraic-lattice conjunct is proved; the $\text{Fp}(D) \hookrightarrow (D \rightarrow D)$ embedding conjunct is false , kernel-checked (r0032). The result is settled, but it is not a proof of the paper’s sentence, so it is counted in neither column	—	—
3	Unnumbered prose claims proved	12	—	—
4	Mathlib foundations reused	12	—	12
5	Theorems in the development	590 live (+6 commented out as unused)	—	—
6	sorry in the development	—	1 : thm18 in Skeleton/Section6.lean	—

Round r0032. Five agents; three landed, two were still running when this was written.

#	Stream	Landed
1	Universality.lean	Lemma 24 and Theorem 25 , with <code>thm25_powerset</code> : $P \ N$ is universal. Proved at <i>cpo</i> strength — no step spends algebraicity or countability, so it is stronger than the paper’s statement
2	IdealCompletion.lean, Powerdomain/BoundedComplete.lean	the idealSup repair , and bounded completeness reinstated for D_b and $D_\#$
3	FinitaryProjectionEmbedding.lean	Theorem 16’s embedding conjunct refuted

Theorem 16’s second conjunct is false. The paper asserts the inclusion $i : Fp(D) \hookrightarrow (D \rightarrow D)$ is an embedding. Against a five-element bifinite domain with two incomparable minimal upper bounds — one that satisfies `thm16`’s first conjunct, so the refutation isolates the second — there are two incomparable maximal finitary projections below $f = \lambda x. m_1$ and no greatest one, while any monotone section would have to produce one. The sketch’s error is exact: $p \sqsubseteq f \iff im(p) \cap K(D) \subseteq S_f$, so what is needed is a normal set **contained in** S_f , whereas the paper takes the least normal set **containing** it. Those agree only when S_f is already normal. No choice of N_f repairs it. A positive statement remains available: the conjunct holds when every S_f has a greatest normal subposet, which bounded complete domains satisfy.

The idealSup repair, closing the third instance of one defect. `idealSup` now branches on `Order.IsIdeal (genIdeal S)` — the ideal *generated by* the union — rather than on the union itself being an ideal. `genIdeal_eq_sUnion_of_isIdeal` proves the two agree wherever the old guard fired, so `lubOfDirected` and `mem_sSup_iff` kept their statements *and their proofs*, and Theorem 11 is byte-identical. `not_boundedComplete_{hoare,smyth,plotkin}` are retired, and `instBoundedCompleteHoare` (no bounded-completeness hypothesis needed) and `instBoundedCompleteSmyth` take their place. The witness is kept as three live lemmas rather than a docstring: the old guard still fails on it, and the repaired `sSup` returns the true least upper bound. A docstring cannot detect a regression.

Round r0029 ran four agents in parallel and closed the definition list.

#	Stream	Landed
1	Powerdomain/Hoare.lean	the Hoare (lower) powerdomain: Pf the finite non-empty subsets of $K(D)$, the lower pre-order, and <code>Domain</code> from Theorem 11
2	Powerdomain/Smyth.lean	the Smyth (upper) powerdomain, same shape, dual orientation
3	Powerdomain/Plotkin.lean	the Plotkin (convex) powerdomain under the Egli–Milner pre-order
4	RecursiveDomain.lean	the recursive domain equation, two formalizations of <i>universal domain</i> , and Theorem 21 — plus <code>recursiveDomain_funSpace</code> , the reflexive domain $D \cong (D \rightarrow D)$

There is no D_∞ to build. Earlier drafts of this inventory listed D_∞ (inverse limit) as an outstanding definition. Reading §7 directly refutes that: the section raises the chain $T_0 \rightarrow e_0 \ T_1 \rightarrow e_1 \ T_2 \rightarrow \dots$, says “This is all very informal, however; how are we to make this idea mathematically precise...?”, and answers with §7.1, *Solving domain equations with closures*. $D \cong D \rightarrow D$ is reached from Theorem 21 and Lemma 23, and the limit is taken inside $Fc(U)$. What stands in D_∞ ’s place is `Recursive.Solves / IsSolvable`, and the two formalizations of universal domain — `IsUniversal` (every domain of the class is a closure of U) and `IsUniversalRetract` (every one is a retract), which the paper states as two different sentences.

Pf is the finite non-empty subsets. The paper defines $Pf(S)$ that way and reserves $\bar{P}f(S)$ for the version including $\emptyset$; all three powerdomains are built over Pf . The distinction is load-bearing and in opposite directions for the three orderings: under the Hoare order $\emptyset \sqsubseteq v$ holds vacuously, so admitting $\emptyset$ would add a point strictly below $\{\perp\}$; under the Smyth order $\emptyset$ is a **top**, so it would add a spurious maximum; under Egli–Milner $\emptyset$ is comparable to nothing but itself, destroying `OrderBot`. The orchestrator’s brief said “finite subsets”; each agent read the PDF and corrected it.

Namespace per agent worked. Every `r0029` declaration lives in `ScottDomains.{Hoare, Smyth, Plotkin, Recursive}`. Four agents, **zero** name collisions — against two in `r0028`, when five agents shared one namespace.

Round r0031 ran four agents and closed the definition list.

#	Stream	Landed
1	ComputableFunction.lean	the paper’s computable function , on Mathlib’s REPred — the last definition
2	Powerdomain/BoundedComplete.lean	Lemma 13 in the paper’s own wording, and a refutation: see the idealSup defect below
3	Powerdomain/Universal.lean	isRepresentable_prod — the product operator is representable over $P\ N$, the hypothesis Lemma 24 lacked ; plus $D \equiv D \times D$
4	Theorem 18	still running at the time of writing

The idealSup defect — the same class, a third time. IdealCompletion.idealSup branches its dte on Order.IsIdeal ($\bigcup_0 \dots$). That is the membership condition for the *union*, but the union is the least upper bound only when the family is **directed**; for a merely **bounded** family the least upper bound is the ideal *generated* by the union. So sSup returns $\perp$ on a bounded non-directed family — kernel-checked in $P\ N$ with the incomparable compacts $\{0\}$ and $\{1\}$, bounded above by $\{0, 1\}$. This follows ScottHom (r0006–r0011) and Smash (r0027), and the r0031 plan asserted the guard was correct, which was wrong. Consequently not_boundedComplete_hoare, _smyth and _plotkin are proved: **no BoundedComplete instance can be claimed for any ideal completion until idealSup is repaired**, by branching on the *generated* ideal rather than on the union being one.

Lemma 13 is nonetheless true and proved, in the paper’s own wording ($BddAbove\ S \rightarrow \exists\ I,\ IsLUB\ S\ I$, with the least element from OrderBot): D_b (Hoare) needs only the union, so it is bounded complete for **every** domain and the hypothesis is never consumed; $D_\#$ (Smyth) spends both hypotheses. There is no lem13_plotkin to prove — Lemma 13 names only $D]$ and $D[$, and Egli–Milner has no join for a bounded pair.

Two inventory rows below are corrected by r0031’s reading of §7. Lemmas 28 and 30 are **not** about $P\ N$ and not about $F_c(U)$: both are stated for *p-representability* over $F_p(U)$, Lemma 28 over §7.3’s ideal completion of dyadic half-open intervals and Lemma 30 over §7.4’s bifinite universal domain. Lemma 23 is therefore **not** Lemma 28’s function-space conjunct. Also, the paper proves $F(X) = X + X$ has **no** representation over $P\ N$, so a + conjunct there is false, not missing.

Theorem 12 is recoverable after all. The row below says its “axioms T” are never legible; pdftotext -layout extracts them cleanly — a continuous algebra $\oplus : E \times E \rightarrow E$ with associativity, commutativity and idempotence, where $T_\#$ adds $s \oplus t \sqsubseteq s$ and T_b adds $s \sqsubseteq s \oplus t$. It belongs to §5.3 with Lemma 13, not to the §7 group.

Round r0028 ran five agents in parallel and roughly doubled the development: $27 \rightarrow 33$ modules, $4440 \rightarrow 8212$ lines, $199 \rightarrow 384$ theorems, $9 \rightarrow 15$ numbered results. Every new result was kernel-audited with #print axioms: all depend only on propext, Classical.choice, Quot.sound, and none on sorryAx.

#	r0028 stream	Landed
1	CoalescedSum.lean, Skeleton/Sum.lean	$D + E$ as a cpo (sumSup, sumCpo), then lem10_sum, lem17_sum, lem17_smash — Lemma 10 at 6 of 6 conjuncts, Lemma 17 at 5 of 5
2	IdealCompletion.lean	Theorem 11 , both halves, on Mathlib’s Order.Ideal; §5 unblocked
3	FinitaryProjectionPoset.lean, Skeleton/Section6b.lean	$F_p(D)$ and $F_c(D)$ as posets, Theorem 16 (algebraic-lattice conjunct), Lemma 20 , and IsClosure.domain_range — Lemma 19 at the paper’s strength

#	r0028 stream	Landed
4	MinimalUpperBounds.lean	minimal upper bounds, U, U^∞ , and <code>isPlotkinOrder_iff_mubClosure</code> — a characterization the paper does not state. Theorem 18 still open
5	UniversalDomain.lean	Theorem 22 and Lemma 23 , with <i>representable</i> defined from §7; opens the route to D^∞

A name clash lake build could not catch. Streams 3 and 5 each defined `IsClosure.apply_sSup_of_directed` and `isClosure_sSup`. The build passed at 971 jobs because no module imported both; the clash appeared the moment anything did — here, an axiom audit importing the pair. `isClosure_sSup` was the *same* statement proved twice; the single copy now lives in `Skeleton/Section6.lean` beside `IsClosure`, which both modules already import, so no call site changed. The two `apply_sSup_of_directeds` were *different* statements sharing a name — one indexed by a set of the subtype $\mathfrak{r}(\text{im } r)$, one by an ambient $D : \text{Set } \alpha$ with $D \subseteq \text{im } r$ — so the subtype form was renamed `IsClosure.apply_sSup_val_image_of_directed`. **A green build is not evidence that parallel work composes**; importing every new module together is.

Row 5 counts lines matching `^(@[...] )?(theorem|lemma)` across the 37 modules.

The sorry burn-down (from r0026). The *sorry*s are deliberate scaffolding: fixed *statements* of outstanding results, confined to `ScottDomains/Skeleton/`, one file per agent worktree, so that three agents can prove them in parallel without any agent editing a declaration another depends on. Every other module remains *sorry*-free, and the count above is the burn-down metric — it goes $10 \rightarrow 0$. Round r0027, run as three agents in parallel, took it **$10 \rightarrow 1$** .

#	Open statement	Result	State after r0027
1	prop15	Prop 15 — every bounded complete domain is bifinite	proved
2	thm18	Thm 18 — $D, D \rightarrow D$ domains $\implies D$ bifinite	sorry — the paper gives no proof, citing Smyth [Smy83a]; the obstacle is recorded in the docstring
3	lem19	Lem 19 — the image of a closure is a domain	proved , via <code>IsClosure.rangeCompletePartialOrder</code>
4–7	lem10_prod, lem10_smash, lem10_lift, lem10_strict	Lem 10 — bounded completeness closed under $\times, \otimes, ()^\perp, \rightarrow^\perp$. The $\rightarrow$ conjunct is already proved (Thm 7’s bounded-complete half, r0007)	all four proved ; Lem 10 is then 5 of 6 conjuncts — $D + E$ is not stated
8–10	lem17_prod, lem17_lift, lem17_fun	Lem 17 — bifiniteness closed under $\times, ()^\perp, \rightarrow$	all three proved ; Lem 17 is then 3 of 5 conjuncts — $D \otimes E$ and $D + E$ are not stated

The + conjuncts, closed in r0028. `CoalescedSum.lean` was 181 lines of ingredients with no `sSup` and no `cpo` instance, so there was no $D + E$ to state a conjunct over. It now has both, and the guard is the membership condition `IsNonBotSum`

(`sumCandidate (sumBase s)`) — the defining predicate of the subtype — not directedness, so the defect fixed in `ScottHom` and then in `Smash` did not recur a third time. One wrinkle the `smash` did not have: $\alpha \otimes \beta$ carries no `SupSet`, so a summand must be selected first; that selection is not a second guard, because a set with an upper bound at all lies in one summand.

The `smashSup` defect (r0027). `lem10_smash` was not merely open: as `smashSup` stood, it was **false**, and the kernel confirmed a refutation. `smashSup` branched its dte on the base being nonempty *and directed*, so a merely **bounded** non-directed base fell to the adjoined $\perp$, which is not even an upper bound. Witness: $D = \text{Prop} \times \text{Prop}$, $E = \text{Prop}$, $s = \{\uparrow((\text{True}, \text{False}), \text{True}), \uparrow((\text{False}, \text{True}), \text{True})\}$, bounded by $\uparrow((\text{True}, \text{True}), \text{True})$. This is the same defect `ScottHom.lean` records having already hit for the function space. The repair branches on the coordinatewise supremum landing in `NonBotPair` — the condition under which it is an element of $D \otimes E$ at all, rather than a merely sufficient condition for it. `smashSup_of_directed` and `smashSup_of_empty` kept their statements and were reproved, which is the agreement claim stated in Lean rather than in prose, and `smashCpo` needed no change.

Kernel check on the r0027 merges (`#print axioms`): all ten proved statements plus `smashCpo`, `smashSup_of_directed` and `smashSup_of_empty` depend only on `propext`, `Classical.choice`, `Quot.sound` — `lem10_prod` and `lem19` do not even need `Classical.choice`. None depends on `sorryAx`. `thm18` does, as its `sorry` requires.

Row 2 counts only the paper’s 30 **numbered** results (Theorems / Lemmas / Proposition 1–30). Row 3 counts the claims the paper makes **in prose** rather than as numbered results; these are paper content too, and all twelve are formally verified:

#	Paper claim	Where	Lean
1	“the compact elements [of $P \rightarrow N$] are just the finite subsets of N ”	p. 9	<code>isCompactElement_iff_finite</code>
2	“ $P \rightarrow N$... is a domain”	p. 9	<code>instance :</code> <code>Domain (Set X)</code>
3	“ $D \rightarrow E$ is a ... cpo”	Thm 7 proof	<code>instance :</code> <code>CompletePartialOrder</code> <code>(ScottHom α β)</code>
4	“... bounded complete ... whenever E is”	Thm 7 proof	<code>instance :</code> <code>BoundedComplete</code> <code>(ScottHom α β)</code>
5	“step(s) ... is continuous”	Thm 7 proof	<code>scottContinuous_stepFun</code>
6	“... and compact in the ordering on $D \rightarrow E$ ”	Thm 7 proof	<code>isCompactElement_step</code>
7	“they form a basis for $D \rightarrow E$ ”	Thm 7 proof	<code>instance :</code> <code>IsAlgebraic</code> <code>(ScottHom α β)</code>
8	every compact function is a <i>finite</i> join of step functions	Thm 7 proof, implicit	<code>exists_finite_isLUB_of_isCo</code>
9	“an embedding is an injection”	§3.1	<code>IsEmbeddingProjectionPair.i</code>
10	“a projection is a surjection”	§3.1	<code>IsEmbeddingProjectionPair.s</code>
11	“it is easy to check that p_N ... is a finitary projection”	§3.1	<code>isFinitaryProjection_normal</code>
12	“the set of strict continuous functions $D \rightarrow E$ is also a cpo”	§2.1	<code>ScottDomains.strictHomCpo</code>

Six of those eleven are the body of **Theorem 7**, which is now **complete** — all four conjuncts of its conclusion (cpo, bounded complete, algebraic, countably based) are formally verified, as `ScottHom.isBoundedCompleteDomain_scottHom`.

It is proved under **weaker hypotheses than the paper states**: bounded completeness of D is never used. D need only be a domain. The function space is a cpo for any preordered D , algebraic when D and E are, bounded complete because E is, and countably based because D and E are.

The remaining theorems are supporting API: the $\ll$ calculus, the `compactsBelow` machinery, the pointwise order and suprema on $D \rightarrow E$, and the step-function adjunction. The paper assumes or elides all of it.

Reference audit (r0020). Of the theorems in the development, 16 are never cited elsewhere. Nine of those are *terminal by design* — they are the paper’s own claims, so nothing should cite them (`injective_embedding`, `surjective_projection`, the `isNormalIn_sUnion*` family and `mono_right` for Lemma 4, `singleton_bot_isNormalIn` for $\{\perp\} \in P(C)$, `isLeast_kleeneFix_le`, `eq_kleeneOperator_op`). The other six were speculative API written for callers that never appeared; they are **commented out in place**, each with a note on why it exists and what is instructive about it, and the build is unchanged — which also confirms that the three `@[simp]` ones among them were never firing implicitly.

The development is **43 modules, 12575 lines, 1 sorry, 0 other warnings** (measured by `scripts/counts.sh`). Counts of definitions, results and theorems are in the Progress table above — they are not repeated here, so that this section cannot drift out of step with it. What each round delivered:

#	Round	Module	Contents
1	r0003	ScottDomains/WayBelow.lean	way-below $\ll$ at $[\text{Preorder } \alpha]$, 7 theorems; $x \ll x \leftrightarrow \text{IsCompactElement } x$ holds by <code>Iff.rfl</code>
2	r0004	ScottDomains/Domain.lean	<code>IsAlgebraic</code> , <code>Domain</code> (with the paper’s countable-basis condition), <code>BoundedComplete</code> , 8 theorems, <code>Domain Prop</code>
3	r0005	ScottDomains/Powerset.lean	the paper’s $P \ N$ (p. 9): compacts of $\text{Set } X$ are exactly the finite subsets, hence <code>Domain (Set $\mathbb{N}$)</code> — the nondegenerate witness
4	r0006–r0007	ScottDomains/ScottHom.lean	the continuous function space $D \rightarrow E$: <code>ScottHom</code> , the pointwise order, <code>CompletePartialOrder</code> , and <code>BoundedComplete</code> when E is — Theorem 7’s first sentence in full
5	r0008	ScottDomains/StepFunction.lean	the single step function <code>step k e</code> : continuity (from k compact), the adjunction $\text{step } k \ e \leq f \leftrightarrow e \leq f \ k$, and compactness in $D \rightarrow E$ (from e compact)
6	r0009	ScottDomains/FunctionSpaceDomain.lean	Domain is algebraic — the paper’s “they form a basis for $D \rightarrow E$ ”; the two halves of <code>IsAlgebraic</code> use disjoint hypotheses (directedness needs only E bounded complete; the lub needs only D, E algebraic)
7	r0010	ScottDomains/CompactFunctionSpace.lean	every compact function is a finite join of step functions — the finiteness half of the basis claim
8	r0011	ScottDomains/FunctionSpaceCountable.lean	<code>K(D, E)</code> is countable, hence <code>Domain (ScottHom $\alpha \ \beta$)</code> — Theorem 7 complete
9	r0012	ScottDomains/NormalSubposet.lean	the normal-subposet relation $\triangleleft$ and Lemma 4 , all four parts
10	r0012–r0013	ScottDomains/Projection.lean	embedding–projection pairs, projections, the paper’s “an embedding is an injection, a projection is a surjection”; $\text{im}(p)$ is a cpo and finitary projections
11	r0014	ScottDomains/FinitaryProjection.lean	Lemma 5 — the compacts of $\text{im}(p)$ are $\text{im}(p) \cap K(D)$ (needs only that p is a projection), and $\text{im}(p) \cap K(D) \triangleleft K(D)$
12	r0015	ScottDomains/NormalProjection.lean	$p_N = \bigsqcup \{y \in N \mid y \sqsubseteq x\}$: continuous, a projection, and $\text{im}(p_N) \cap K(D) = N$ — half of Theorem 6 ’s correspondence, plus order preservation both ways
13	r0016	ScottDomains/Theorem6.lean	Theorem 6 — p_N is finitary, $p_{\{\text{im}(p) \cap K(D)\}} = p$, and the correspondence assembled
14	r0017	ScottDomains/FixedPoint.lean	Theorem 1 — $\bigsqcup_n f^n(\perp)$ is the least fixed point of a continuous f on a cpo. Not Mathlib reuse; see the §2 table

#	Round	Module	Contents
15	r0018	ScottDomains/UniformFixedPoint	Theorem 3 — fix is the unique uniform fixed-point operator; $\downarrow a$ as a cpo
16	r0019	ScottDomains/Product.lean	$D \times E$ as a cpo (the one construction needing no case split) and Lemma 8 parts 1–3
17	r0021	ScottDomains/Currying.lean	Lemma 8.4 — currying, $D \rightarrow (E \rightarrow F) \cong (D \times E) \rightarrow F$, and with it Lemma 8 complete
18	r0022	ScottDomains/EffectivePresentation	§2.1 effective presentation — the enumeration of the basis with its two decidability conditions
19	r0023	ScottDomains/Lift.lean	the lift $D \downarrow$ as a cpo, on Mathlib’s WithBot
20	r0024	ScottDomains/StrictHom.lean	the strict function space $D \rightarrow_{\perp} E$ as a cpo — needs no case split, since both branches of ScottHom’s sSup are strict
21	r0025	ScottDomains/Smash.lean	§4.3’s smash product $D \otimes E$ — the non-bottom pairs with a new bottom adjoined, as a cpo
22	r0025	ScottDomains/Bifinite.lean	§6.1’s Plotkin order and bifinite domain

§2 and §3 are now complete — the only §3 omission is the paper’s *computable function*, which needs an r.e.-predicate notion Mathlib does not supply.

Dependency structure of the remaining definitions

#	Definition	Prerequisites	Status
1	smash product $D \otimes E$	Domain, OrderBot	✓ done, r0025
2	bifinite / Plotkin order	IsNormalIn ($\triangleleft$, r0012)	✓ done, r0025
3	sum $D + E$	Domain	independent, ready
4	D_{∞}	embedding–projection pairs (r0012)	independent, but large (inverse limits)
5	the three powerdomains	the ideal completion (Theorem 11) — <i>not</i> built	blocked ; the three are independent of each other once it exists

Rows 1 and 2 were written as two modules and checked in a single build, which is the parallelism the structure actually admits — the bottleneck is the build-and-fix loop, not the writing.

Next: the sum $D + E$, then Theorem 11 to unblock the powerdomains, then Lemma 9 (the strict analogues of Lemma 8) and Lemma 10 (closure of bounded completeness).

Work counts

- **Reuse from Mathlib — no work (11):** poset, directed, cpo, $\perp$, monotone, continuous, fixed-point operator, Schröder–Bernstein (2), algebraic lattice, product, λ -notation. (Theorem 1 was listed here in an earlier draft and has been removed: Mathlib’s OrderHom.lfp is Knaster–Tarski over a complete lattice, not Kleene’s $\bigsqcup_n f^n(\perp)$ over a cpo. It is proved in ScottDomains/FixedPoint.lean, so **29** numbered results needed proof, not 28.)
- **Generalize / adapt (4):** compact element (IsCompactElement — its *definition* is already stated at [PartialOrder α] and so applies to a dcpo verbatim; what is CompleteLattice-only is every lemma about it, so the dcpo API must be re-proved), sum, lift (WithBot/Sum), ideal completion (Order.Ideal).
- **Definitions to define — new (≈ 13 ; 4 done in r0003–r0004, 9 remaining):** way-below $\ll$ (**done** — ScottDomains/WayBelow.lean), algebraic cpo, **domain**, bounded-complete (**all three done** — ScottDomains/Domain.lean),

embedding–projection pair, (finitary) projection, normal subposet, effective presentation, smash product, the three powerdomains (Hoare / Smyth / Plotkin), bifinite / Plotkin order, and D^∞ .

- **Theorems to prove (28):** 28 of the paper’s 30 numbered results — Theorems 3, 6, 7, 11, 12, 14, 16, 18, 21, 22, 25, 26, 27, 29; Lemmas 4, 5, 8–10, 13, 17, 19, 20, 23, 24, 28, 30; Proposition 15. Only Theorems 1 & 2 come free from Mathlib.

Bottom line: ≈ 13 definitions to define + 28 results to prove, on top of 12 reused Mathlib foundations. After r0004: **9 definitions + 28 results** remain.

Lean column legend: ✓ reuse Mathlib (name given) · ~ partial (Mathlib has a related or lattice-only version — generalize) · ✕ define / prove (absent from Mathlib v4.32.2, confirmed by grep).

Statements are paraphrased/de-garbled from the PDF (1990 Type-3 fonts render $\rightarrow$ as $!$, drop fi ligatures, mangle math), so read them as a guide to *what* each result is. Notation: $\sqsubseteq$ order, $\sqcup$ directed sup, $\perp$ bottom, $\ll$ way-below, $K(D)$ compacts, $Fp(D)$ finitary projections.

§2 Recursive definitions of functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
2.1	Def	—	poset (partially ordered set)	✓ <code>PartialOrder</code>
2.1	Def	—	directed subset M : every finite $u \subseteq M$ has an upper bound in M	✓ <code>DirectedOn</code> / <code>IsDirected</code>
2.1	Def	—	cpo : poset in which every directed M has a lub $\sqcup M$	✓ <code>CompletePartialOrder</code> (chains: <code>OmegaCompletePartialOrder</code>)
2.1	Def	—	bottom $\perp$ (least element)	✓ <code>OrderBot</code>
2.1	Def	—	monotone function	✓ <code>Monotone</code> / <code>OrderHom</code>
2.1	Def	—	continuous : monotone, $f(\sqcup M) = \sqcup f(M)$ for directed M	✓ <code>ScottContinuous</code>
2.1	Thm	1	Fixed-Point Theorem : $f : D \rightarrow D$ continuous $\implies$ least fixed point $\sqcup_n f^n(\perp)$	✓ proved (r0017) — <code>ScottDomains.theorem1</code> . Not Mathlib reuse: <code>OrderHom.lfp</code> is Knaster–Tarski over a <i>complete lattice</i> with only monotonicity, a different theorem
2.2	Thm	2	Schröder–Bernstein for sets	✓ <code>Function.schroeder_bernstein</code> (<code>SetTheory/Cardinal/SchroederBernstein.lean</code>) — the name in an earlier draft of this row, <code>Function.Embedding.schroederBernstein</code> , does not exist
2.3	Def	—	fixed-point operator (uniform)	✓ <code>OrderHom.lfp</code> / <code>LawfulFix</code>
2.3	Thm	3	The standard operator is the unique uniform fixed-point operator	✓ proved (r0018) — <code>ScottDomains.theorem3</code> ; a fixed point operator is formalized as a family over every <code>CompletePartialOrder</code> in a universe

§3 Effectively presented domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	compact element x : $x \sqsubseteq \sqcup M(\text{dir.}) \implies x \sqsubseteq y$ some $y \in M$; $K(D)$	~ <code>IsCompactElement</code> — def is <code>[PartialOrder]</code> , so reusable on a <code>dcpo</code> ; its lemmas are all <code>CompleteLattice</code> -only
3.1	Def	—	way-below $\ll$ (approximation relation behind compactness)	✓ <code>ScottDomains.WayBelow</code> (r0003) — $x \ll x \leftrightarrow \text{IsCompactElement } x$ by <code>Iff.rfl</code>

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	algebraic cpo: $x = \bigsqcup \{x' \in K(D) : x' \sqsubseteq x\}$ (directed)	✓ <code>ScottDomains.IsAlgebraic</code> (r0004)
3.1	Def	—	domain = algebraic cpo whose basis $K(D)$ is countable (the paper’s definition, p. 9 — the countability condition was missing from an earlier draft of this row)	✓ <code>ScottDomains.Domain</code> (r0004)
3.1	Def	—	bounded complete : $\perp +$ every bounded subset has a sup — a <i>separate</i> predicate; the paper composes them as “bounded complete domain” (Thm 7, Lem 10, Lem 13, Thm 14), which is the literature’s <i>Scott domain</i>	✓ <code>ScottDomains.BoundedComplete</code> (r0004); the compound is <code>[Domain α] [BoundedComplete α]</code>
3.1	Def	—	(countably based) algebraic lattice	✓ <code>IsCompactlyGenerated</code> (+ <code>CompleteLattice</code>)
3.1	Def	—	embedding–projection pair (g, f)	✓ <code>ScottHom.IsEmbeddingProjectionPair</code> (r0012)
3.1	Def	—	projection; finitary projection p : $p \circ p = p \sqsubseteq \text{id}$, $\text{im}(p)$ a domain	✓ <code>ScottHom.IsProjection</code> (r0012), <code>ScottHom.IsFinitaryProjection</code> (r0013) — $\text{im}(p)$ carries a <code>CompletePartialOrder</code> via <code>IsProjection.rangeCompletePartialOrder</code>
3.1	Def	—	normal subposet / substructure	✓ <code>ScottDomains.IsNormalIn</code> , notation $\triangleleft$ (r0012)
3.1	Lem	4	$\langle P(C), \triangleleft \rangle$ of substructures is a cpo with $\{\perp\}$ least	✓ proved (r0012), all four parts
3.1	Lem	5	p finitary projection $\implies$ compacts of $\text{im}(p)$ are $\text{im}(p) \cap K(D)$, and $\text{im}(p) \cap K(D) \triangleleft K(D)$	✓ proved (r0014) — <code>IsProjection.isCompactElement_iff</code> (needs only <i>projection</i>) and <code>IsFinitaryProjection.isNormalIn_compacts</code>
3.1	Thm	6	Isomorphism: normal substructures $\cong \text{Fp}(D)$ (finitary projections)	✓ proved (r0015–r0016) — <code>ScottDomains.theorem6</code>
3.1	Thm	7	D, E bounded-complete domains $\implies D \rightarrow E$ bounded-complete domain	✓ proved (r0006–r0011) — <code>ScottHom.isBoundedCompleteDomain_scottHom</code> ; D bounded complete is not needed
3.2	Def	—	effective presentation $d : \mathbb{N} \rightarrow K(D)$; effectively presented domain	✓ <code>ScottDomains.EffectivePresentation</code> (r0022). The paper’s computable function is ✓ defined (r0031) in <code>ComputableFunction.lean</code> , on Mathlib’s REPred (<code>Mathlib/Computability/RE.lean:157</code>). An earlier draft of this row said “Mathlib v4.32.2 has no <code>RePred</code> or equivalent (grep finds none)” — that grep used the wrong capitalization, and the claim was false. Two caveats recorded there: <code>decidableLE</code> is too weak to prove anything computable (a <code>Decidable</code> instance may be <code>Classical.dec</code>), so results need an explicit <code>RecursiveLE</code> hypothesis; and composition awaits <code>REPred</code> closure under $\wedge$ and $\exists$, which Mathlib’s five-lemma API does not supply

§4 Operators and functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.1	Def	—	product $D \times E$	✓ Prod order from Mathlib; the cpo instance is <code>ScottDomains.instCompletePartialOrderProd</code> (r0019) — Mathlib has <code>Prod.supSet</code> and <code>isLUB_prod</code> but no cpo instance
4.2	Def	—	Church’s λ-notation (continuous abstraction)	✓ <code>OrderHom / ωCPO ContinuousHom</code>
4.3	Def	—	smash product $D \otimes E$	✓ <code>ScottDomains.Smash + smashCpo</code> (r0025)
4.4	Def	—	sum $D + E$; lift $D \perp$	both ✓ — <code>lift ScottDomains.liftCpo</code> on <code>WithBot</code> (r0023); the coalesced sum <code>CoalescedSum</code> with <code>sumSup</code> and <code>sumCpo</code> (r0028), its <code>sSup</code> guarded on landing in <code>NonBotSum</code>
4.x	Lem	8	$D \times E \cong E \times D$; $(D \times E) \times F \cong D \times (E \times F)$; $D \rightarrow (E \times F) \cong (D \rightarrow E) \times (D \rightarrow F)$; $D \rightarrow (E \rightarrow F) \cong (D \times E) \rightarrow F$	✓ proved — <code>prodComm</code> , <code>prodAssoc</code> , <code>scottHomProd</code> (r0019); <code>scottHomCurry</code> (r0021)
4.x	Lem	9	Product/function-space iso laws over D, E, F	✗ not statable — the PDF drops every $\otimes$ and $\perp$, so which operators the laws range over is unreadable. Statement recovery is r0030 agent5’s assignment
4.5	Lem	10	D, E bounded complete $\implies \rightarrow, \times, \otimes, +, () \perp$ bounded complete	✓ proved, 6 of 6 conjuncts — $\rightarrow$ r0007; $\times, \otimes, () \perp, \rightarrow \perp$ r0027 (<code>Skeleton/Lemma10.lean</code>); $+$ r0028 (<code>Skeleton/Sum.lean</code>)
4.5	Thm	11	Ideal completion of a countable pre-order is a domain (all domains so arise)	✓ proved (r0028) — <code>IdealCompletion.thm11</code> and <code>thm11_converse</code> , on Mathlib’s <code>Order.Ideal</code>
4.5	Thm	12	Initiality of a continuous algebra satisfying axioms T	✗ prove — statable after all (r0031): the T axioms extract cleanly as a continuous $\otimes$ with associativity, commutativity, idempotence, plus $s \otimes t \sqsubseteq s$ for $T \#$ and $s \sqsubseteq s \otimes t$ for $T \flat$. It sits in §5.3 with Lemma 13
4.5	Lem	13	D bounded complete $\implies$ powerdomains $D], D[$ bounded complete	✓ proved (r0031) — <code>PowerdomainBC.lem13_hoare</code> , <code>lem13_smyth</code> , in the paper’s wording. $D \flat$ needs no bounded-completeness hypothesis at all; there is no convex conjunct to prove
4.5	Thm	14	Equivalent characterizations of an (algebraic/BC) domain	✗ not statable — the list of characterizations is garbled. Statement recovery is r0030 agent5’s assignment

§5 Powerdomains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
5.1	Def	—	powerdomain (non-deterministic outcomes)	✓ each of the three is <code>IdealCompletion (Pf K(D))</code> under its pre-order (r0029)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
5.2	Def	—	Hoare (lower), Smyth (upper), Plotkin (convex) powerdomains	✓ ScottDomains.Hoare.Powerdomain, Smyth.Powerdomain, Plotkin.Powerdomain (r0029), each with its Domain instance from Theorem 11 and its compacts characterized as the principal ideals
5.3	—	—	Universal & closure properties (see Lem 13, 28, 30)	✗ prove — r0030 wave A

§6 Bifinite (SFP) domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.1	Def	—	Plotkin order / bifinite (SFP) domain	✓ ScottDomains.IsPlotkinOrder, IsBifinite (r0025)
6.1	Prop	15	Every bounded-complete domain is bifinite	✓ proved (r0027) — ScottDomains.prop15, the paper's own proof over lubClosure u
6.1	Thm	16	D bifinite $\implies Fp(D)$ is an algebraic lattice	settled, half each way: the algebraic-lattice conjunct ✓ proved (r0028) as ScottDomains.thm16; the $Fp(D) \hookrightarrow (D \rightarrow D)$ embedding conjunct ✗ refuted (r0032) in FinitaryProjectionEmbedding.lean, kernel-checked, with the sketch's exact error identified — see the note under Progress
6.2	Lem	17	D, E bifinite $\implies \rightarrow, \times, \otimes, +, ()^\perp$ bifinite (incl. function space)	✓ proved, 5 of 5 conjuncts — $\times, ()^\perp$, $\rightarrow$ r0027 (Skeleton/Lemma17.lean); $\otimes$, $+$ r0028 (Skeleton/Sum.lean)
6.2	Thm	18	If D and $D \rightarrow D$ are domains, then D is bifinite	✗ prove — the development's only sorry . Reduced by isBifinite_iff_mubClosure (r0028) to two obligations; blocked on a constructor for continuous functions on a domain that is not bounded complete
6.2	Lem	19	closure $r:D \rightarrow D$ ($r \circ r = r \sqcap id$) $\implies im(r)$ is a domain	✓ proved — lem19 (r0027) gives the cpo structure; IsClosure.domain_range (r0028) gives the paper's full strength, $im(r)$ a domain with basis $\{r(k) \mid k \in K(D)\}$
6.2	Lem	20	D domain $\implies Fc(D)$ (finitary closures) is a cpo	✓ proved (r0028) — ScottDomains.lem20, over $Fc \ \alpha$ with the pointwise order

§7 Recursive definitions of domains (universal domain, D_∞)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Def	—	recursive domain equation; universal domain. (Earlier drafts of this row said “ D_∞ (inverse limit)” — §7 builds no inverse limit; see the note under Progress)	✓ Recursive.Solves / IsSolvable, and Recursive.IsUniversal / IsUniversalRetract for the paper’s two phrasings (r0029)
7	Thm	21	F representable over cpo U $\implies$ a domain D with $D \cong F(D)$	✓ proved (r0029) — ScottDomains.Recursive.thm21; with IsRepresentable ₂ .diag and Lemma 23 it yields recursiveDomain_funSpace, the reflexive domain $D \cong (D \rightarrow D)$
7	Thm	22	Any countably-based algebraic lattice L: a closure $r : P(\mathbb{N}) \rightarrow L$	✓ proved (r0028) — ScottDomains.thm22, with thm22_of_isCompactlyGenerated the Mathlib-vocabulary form
7	Lem	23	The function-space operator is representable over $P(\mathbb{N})$	✓ proved (r0028) — ScottDomains.lem23
7	Lem	24	U cpo; $\times$ and $\rightarrow$ representable over U $\implies$ (setup for universality)	✓ proved (r0032) — Universality.lem24, concluding also that D, E are closures of U, which its own proof needs
7	Thm	25	U non-trivial domain representing $\times, \rightarrow \implies$ U universal	✓ proved (r0032) — Universality.thm25, with thm25_powerset instantiating it at P N and thm25_isUniversal in Recursive.IsUniversal form. Proved at cpo strength: no step spends algebraicity or countability, so it is stronger than the paper’s statement
7	Thm	26	Any signature $(s_1, \dots, s_n)$: combinators $F_1, \dots, F_n$ solving the equations	✗ prove
7	Thm	27	Any bounded-complete D: a projection of the universal domain onto D	✗ prove
7	Lem	28	Operators $\rightarrow, \times, \otimes, +, ()^\perp, ()^\downarrow, ()^\uparrow$ representable over U	✗ prove
7	Thm	29	D bifinite $\implies$ D+ bifinite; solving $D \cong D+$	✗ prove
7	Lem	30	Operators $\rightarrow, \times, \otimes, +, ()^\perp, ()^\downarrow, ()^\uparrow$ p-representable over universal bifinite V	✗ prove

Tally (matched): ✓ reuse Mathlib $\approx$ **12** (poset, directed, cpo, $\perp$, monotone, continuous, fixed-point Thm 1 & operator, Schröder–Bernstein, algebraic lattice, product, λ -notation) · $\sim$ partial $\approx$ **4** (compact element, sum/lift, ideal completion Thm 11, Thm 22) · $\times$ define / prove $\approx$ **44**, of which 4 are done ($\ll$ r0003; algebraic, domain, bounded-complete r0004) $\rightarrow$ **40 remaining** — the bifinite / powerdomain / D_∞ development and all 28 numbered results.

What is done, and what is next, is listed under **Progress** at the top of this file.